

Scientific Programming 1 - Practice Exams

This document contains **two practice exams**. Each has the same form as the real exam: three assignments, to be made in two hours.

- Make the practice exams in the online exam editor, linked from the course website (sp.proglab.nl).
- Put the solutions of all six assignments in the single file `sp1_practice_exam.py`. It is already open in the editor.
- The Checkpy button checks your solutions.
- You may use the course website as a resource, and a printed version of the cheatsheet. The same goes for the real exam: there too you may use the course website (the cheatsheet is on it as well) and a printed cheatsheet.
- Do not use external modules such as `numpy`, `csv`, or others unless the assignment specifically says you can.
- You are evaluated solely on the correctness of your solutions. Code design, comments, and style are not important.

Practice Exam 1

Assignment 1: Collatz

Write a function `collatz(number)` that returns the Collatz sequence of `number`. Each next number in the sequence is found from the previous one:

- if the number is odd, multiply it by 3 and add 1;
- if the number is even, divide it by 2.

The sequence always reaches 1 in the end, and then it stops. The starting number itself is not part of the sequence.

Example Usage

```
print(collatz(3))
print(collatz(16))
```

Expected Output

```
[10, 5, 16, 8, 4, 2, 1]
[8, 4, 2, 1]
```

Tip: use integer division (`//`), so your numbers stay integers instead of becoming floats.

Assignment 2: Swap

Write a function `swap_words(text)` that returns a new string in which the first word of `text` is swapped with the second, the third with the fourth, and so on. If the text has an odd number of words, the last word stays where it is.

Example Usage

```
print(swap_words("Why is a raven like a writing desk?"))
print(swap_words("You can always take more than nothing."))
```

Expected Output

```
is Why raven a a like desk? writing
can You take always than more nothing.
```

Assignment 3: Word Lengths

Write a function `words_of_length(words, lengths)` that takes a list of strings `words` and a list of numbers `lengths`. It returns a new list with only the words whose length is equal to one of the numbers in `lengths`, in the same order as in `words`.

You are **not allowed** to use the `in` operator to check whether a length occurs in `lengths` (so no `if len(word) in lengths`). Use a loop instead.

Example Usage

```
words = ["one", "two", "three", "four", "five", "six", "seven"]
print(words_of_length(words, [5, 3]))
```

```
fruits = ["apple", "fig", "banana", "kiwi", "plum", "cherry", "date"]
print(words_of_length(fruits, [6, 4]))
```

Expected Output

```
['one', 'two', 'three', 'six', 'seven']
['banana', 'kiwi', 'plum', 'cherry', 'date']
```

Practice Exam 2

Assignment 1: Bounce

Write a function `bounce(n)` that returns a list of `n` numbers. The list starts with `1.0`, and each next number is found from the previous one:

- On the way up, multiply the number by 4. As soon as a number is 100 or higher, turn around.
- On the way down, divide the number by 2. As soon as a number is lower than 2, turn around again.

Example Usage

```
print(bounce(20))
```

Expected Output

```
[1.0, 4.0, 16.0, 64.0, 256.0, 128.0, 64.0, 32.0, 16.0, 8.0, 4.0, 2.0,
1.0, 4.0, 16.0, 64.0, 256.0, 128.0, 64.0, 32.0]
```

Assignment 2: Gregory-Leibniz

The Gregory-Leibniz series approximates pi:

$$\pi \approx \frac{4}{1} - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} + \frac{4}{13} - \dots$$

Each step adds or subtracts a term, and the terms get smaller. The table shows the first four steps, with the approximation after each step:

step	term	approximation
1	take 4/1	4.0
2	subtract 4/3	2.6667
3	add 4/5	3.4667
4	subtract 4/7	2.8952

And so on: the divisor goes up by 2 at each step, and adding and subtracting alternate. You can use the table to check your function for small numbers of steps.

Write a function `gregory_leibniz(n)` that approximates pi in `n` steps.

Example Usage

```
print(gregory_leibniz(1))
print(gregory_leibniz(10))
print(gregory_leibniz(1000000))
```

Expected Output

```
4.0
3.0418396189294032
3.1415916535897743
```

Assignment 3: Home Advantage

For this assignment you need the files `barca.txt` and `barca_short.txt`. They contain the results of the football matches of F.C. Barcelona in the seasons 2011/12 to 2013/14, and look like this:

```
29/08/11,Villarreal,won,5,0,home
10/09/11,Sociedad,draw,2,2,away
17/09/11,Osasuna,won,8,0,home
...
03/05/14,Getafe,draw,2,2,home
11/05/14,Elche,draw,0,0,away
17/05/14,Ath Madrid,draw,1,1,home
```

The fields are separated by commas and contain:

1. the date of the match;
2. the opponent;
3. the result: won, lost or draw;
4. the number of goals for Barcelona;
5. the number of goals for the opponent;
6. the location: home or away.

Playing at home is considered an advantage for any football club. Let's see if this is true for Barcelona. Write a function `home_advantage(filename)` that returns the number of home matches won minus the number of away matches won. So a positive number means that more home matches were won than away matches.

Example Usage

```
print(home_advantage("barca_short.txt"))
print(home_advantage("barca.txt"))
```

Expected Output

```
2
15
```